Nahrávání a transformace dat

Zpracovávání výzkumných dat pomocí jazyka R

WebR Status

 Ready!

Program

- opakování z minulé hodiny
- kontrola domácího úkolu
- pipes
- nahrávání dat
- transformace dat
  - úprava jmen sloupců
  - chybějící hodnoty
  - mutace a řazení podle jednotlivých hodnot
  - vytváření sloupců

Kontrola domácí práce

Úloha 1

Vyber z následujícího vektoru každou pátou hodnotu.

1. Pojmenujte si vektor pro snažší manipulaci.

```
 1  hodnoty <- seq(15.2, 45.7, 0.56); hodnoty
```

```
 [1] 15.20 15.76 16.32 16.88 17.44 18.00 18.56 19.12 19.68 20.24 20.80 21.36
[13] 21.92 22.48 23.04 23.60 24.16 24.72 25.28 25.84 26.40 26.96 27.52 28.08
[25] 28.64 29.20 29.76 30.32 30.88 31.44 32.00 32.56 33.12 33.68 34.24 34.80
[37] 35.36 35.92 36.48 37.04 37.60 38.16 38.72 39.28 39.84 40.40 40.96 41.52
[49] 42.08 42.64 43.20 43.76 44.32 44.88 45.44
```

2. Vytvořte sekvenci, která v sobě bude obsahovat čísla v intervalu pěti od 0 do posledního čísla vektoru = každé páté číslo.

```
1  seq(0, length(hodnoty), by = 5)
```

```
[1]  0  5 10 15 20 25 30 35 40 45 50 55
```

3. Použijte výslednou sekvenci čísel k indexaci.

```
1  hodnoty[seq(0, length(hodnoty), by = 5)]
```

```
[1] 17.44 20.24 23.04 25.84 28.64 31.44 34.24 37.04 39.84 42.64 45.44
```

Úloha 2

Z datasetu (co je nahraný v R) jménem `airquality`, který obsahuje informace o měření kvality vzduchu v NYC, vyber hodnoty ze sloupce `Temp` z prvního týdne v květnu.

1. Zobrazme si prvních několik řádků a podívejme se na strukturu.

```
1  str(airquality)
```

```
'data.frame':   153 obs. of  6 variables:
 $ Ozone  : int  41 36 12 18 NA 28 23 19 8 NA ...
 $ Solar.R: int  190 118 149 313 NA NA 299 99 19 194 ...
 $ Wind   : num  7.4 8 12.6 11.5 14.3 14.9 8.6 13.8 20.1 8.6 ...
 $ Temp   : int  67 72 74 62 56 66 65 59 61 69 ...
 $ Month  : int  5 5 5 5 5 5 5 5 5 5 ...
 $ Day    : int  1 2 3 4 5 6 7 8 9 10 ...
```

```
1  head(airquality)
```

	Ozone	Solar.R	Wind	Temp	Month	Day
1	41	190	7.4	67	5	1
2	36	118	8.0	72	5	2
3	12	149	12.6	74	5	3
4	18	313	11.5	62	5	4
5	NA	NA	14.3	56	5	5
6	28	NA	14.9	66	5	6

2. Vyfiltrujeme hodnoty, které byly naměřeny první týden v květnu.

```
#vyber z hodnot sloupce
airquality$Temp[airquality$Month == 5 & airquality$Day %in% 1:7]
```

```
[1] 67 72 74 62 56 66 65
```

```
airquality$Temp[airquality$Month == 5 & airquality$Day <= 7]
```

```
[1] 67 72 74 62 56 66 65
```

```
#vyber z datasetu
airquality[airquality$Month == 5 & airquality$Day %in% 1:7, "Temp"]
```

```
[1] 67 72 74 62 56 66 65
```

3. Lze použít i funkci `filter`, zde už je ale třeba přemýšlet o tom, jestli nám na takto malém kusu kódu opravdu ušetří práci.

```
library(dplyr)
datumy <- filter(airquality,
   Day %in% 1:7,
   Month == 5)
```

Úloha 3

K datasetu `airquality` přidejte nový sloupec s názvem `Weather` který bude obsahovat informaci (text `hot`), zda hodnoty ve sloupci `Temp` jsou vyšší než 70. Pokud není vyšší jako 70, bude obsahovat text `cold`.

1. Pomocí symbolu `$` vytvoříme nový sloupec `Weather`, a přiřadíme do něj hodnotu cold.

```
1  airquality$Weather <- "cold"
2  airquality
```

```
    Ozone Solar.R Wind Temp Month Day Weather
1      41     190  7.4   67     5   1    cold
2      36     118  8.0   72     5   2    cold
3      12     149 12.6   74     5   3    cold
4      18     313 11.5   62     5   4    cold
5      NA      NA 14.3   56     5   5    cold
6      28      NA 14.9   66     5   6    cold
7      23     299  8.6   65     5   7    cold
8      19      99 13.8   59     5   8    cold
9       8      19 20.1   61     5   9    cold
10     NA     194  8.6   69     5  10    cold
11      7      NA  6.9   74     5  11    cold
12     16     256  9.7   69     5  12    cold
13     11     290  9.2   66     5  13    cold
```

Bonus

Pokuste se zjistit, jak si v nápovědě můžete najít seznam nahraných datasetů v R (`iris`, `cars`, `dplyr::starwars` a jiné)

Pro vyhledání můžete použít následující funkci:

```
1 data()
```

Díky této funkci si můžeme rychle zjisti, jaké datasety máme k dispozici v nahraných balíčcích, a **které je proto možné rovnou použít bez jejich opakovaného ukládání do globálního prostředí.**

Jak jsme si ukázali v předchozí lekci, **v R se často kód "vrství"; výsledek jedné funkce se používá jako data pro další výpočet v jiné funkci**.

To lze provést bud´ tak, že se celý výpočet rovnou použije v následující funkci, či se uloží jako proměnná, která se následně použije.

Ukázka:

```
1 #1
2 head(select(filter(iris, Species == "setosa"), Petal.Width))
```

```
  Petal.Width
1         0.2
2         0.2
3         0.2
4         0.2
5         0.2
6         0.4
```

```
1 #2
```

```r
filtrIris <- filter(iris, Species == "setosa")
vyberFiltrIris <- select(filtrIris, Petal.Width)
head(vyberFiltrIris)
```

```
  Petal.Width
1         0.2
2         0.2
3         0.2
4         0.2
5         0.2
6         0.4
```

1. První přístup v sobě nese riziko, že takto psané “funkce ve funkci ve funkci…” **řetězce budou nepřehledné a že uživatel snadno zapomene na jednu ze závorek**.
2. Konstatním ukládáním v druhém případě se nejenom zbytečně zabírá čas a paměť R, ale hlavně pokud dojde ke změně v uložených hodnotách (například, že hledáme `Petal.Length` místo `Petal.Width`), tak **bez opakovaného nahrání následujících řádků kódu co definují následně uložené proměnné by se změny nepropsaly do zbytku kódu**.

Jak tento problém vyřešit?



Pipes

Nástroj, který transportuje data z jedné funkce do druhé jako první argument.

```
filter(iris, Petal.Width > 2) %>%
  select(Petal.Length, Petal.Width) %>%
  head()
```

```
  Petal.Length Petal.Width
1          6.0         2.5
2          5.9         2.1
3          5.8         2.2
4          6.6         2.1
5          6.1         2.5
6          5.5         2.1
```

data %>% funkce(data přivedená skrz pipe, nemusíme je vypisovat, **druhý a další argumenty = definované námi)**

```
head(cars)
```

```
  speed dist
```

```
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

```r
cars %>% head()
```

```
  speed dist
1     4    2
2     4   10
3     7    4
4     7   22
5     8   16
6     9   10
```

Co když chci využít **data v jiném než v prvním argumentu**?

```
1  iris %>%
2    filter(Petal.Length > 0.8, Petal.Width < 1) %>%
3    select(Sepal.Length) %>%
4    unlist() %>% #preved na vektor
5    mean() %>%
6    round(digits = 3, x = .) #ukazka, bezne je x prvni argument
```

```
[1] 5.006
```

> **Specifikování argumentu kam má pipe vyústit**
>
> Tečka u pipe `%>%` **značí místo, kam se mají data vložit**; v některých případech totiž první argument nebývají samotná data.

A co **běžná indexace**? Jak určíme že se mají data vybírat, když se nejedná o argumenty?

```
1  #tecka znaci misto vyusteni pipe
2  cars %>% .[,2]
```

```
 [1]   2  10   4  22  16  10  18  26  34  17  28  14  20  24  28  26  34  34  46
```

```
[20]  26  36  60  80  20  26  54  32  40  32  40  50  42  56  76  84  36  46  68
[39]  32  48  52  56  64  66  54  70  92  93 120  85
```

```
1 cars %>% .$speed
```

```
 [1]  4  4  7  7  8  9 10 10 10 11 11 12 12 12 12 13 13 13 13 14 14 14 14 15 15
[26] 15 16 16 17 17 17 18 18 18 18 19 19 19 20 20 20 20 20 22 23 24 24 24 24 25
```

```
1 cars %>% .["speed"]
```

```
   speed
1      4
2      4
3      7
4      7
5      8
6      9
7     10
8     10
9     10
10    11
11    11
12    12
13    12
```

```
1 cars %>% .[["dist"]]
```

```
 [1]   2  10   4  22  16  10  18  26  34  17  28  14  20  24  28  26  34  34  46
[20]  26  36  60  80  20  26  54  32  40  32  40  50  42  56  76  84  36  46  68
[39]  32  48  52  56  64  66  54  70  92  93 120  85
```

Rozdíl mezi %>% a |>

%>%

- déle používaná, větší šance že na ni narazíte
- pro její používání je potřeba mít načtený balíček `dplyr`
- jako placeholder umístění dat používá tečku .
- funguje s i bez závorkami za názvem funkce

```
1  c(1:20, 15) %>% mean()
```

```
[1] 10.71429
```

```
1  c(9:100, 1:15) %>% mean
```

```
[1] 47.98131
```

|>

- od roku 2021 součástí baseR
- není potřeba mít spuštěný balíček navíc
- jako placeholder umístění dat používá podtržítko _
- vždy je potřeba za název funkce umístit kulaté závorky

```
1  c(2:546, 467, 87) %>% mean()
```

```
[1] 274.011
```

Procvičení

Zjistěte, jaká byla nejmenší váha kuřete krmeného sójovými boby, jejichž data naleznete v datasetu `chickwts`.

Run Code

```
1
```

Jaká byla největší hloubka epicentra zemětřesení z datasetu `quakes`, které bylo větší než medián magnitudy všech naměřených zemětřesení?

Run Code

```
1
```

Pipes a dobrá praxe

- používejte v jednom scriptu pokud možno jeden typ pipe
- pipe má smysl od tří a výše funkcí
- pokud používáte `%>%` dejte si pozor na záměnu tečky a čárky
- jakmile máte deset a více pipes, tak už by bylo přeci jen dobré si to rozdělit
- pipes jsou lineární; pokud pracujete s více jak jedním výstupem, buď to budete mít krkolomné nebo vám kód vůbec nebude fungovat

Nahrávání dat

Nahrávání dat je jedna z klíčových znalostí, které je potřeba při práci v R znát.

Stejně jako u indexace, tak i zde existují jak základní příkazy v `baseR`, tak i funkce ve secializovaných balíčcích (k tomu ale později).

Nastavení working directory

Před nahráváním dat se ujistěte, že máte nastavenou cestu do adresáře, ze kterého R může čerpat soubory.

Pokud máte vytvořený R projekt, který se nachází ve stejné složce jako soubory, tato starost vám odpadá.

funkce `read.table`

Základní funkce k načítání tabulkových dat do R.

Má v sobě několik podtypů pro specifické formáty:

- `read.table` = hodnoty oddělené mezerou
- `read.csv` = hodnoty oddělené čárkou
- `read.delim` = hodnoty oddělené tabulátorem

Co to znamená, že jsou hodnoty oddělené?

Jak má R se souborem pracovat?

Všechny funkce, které nahrávají data do R, v sobě obsahují argumenty, pomocí kterých určujeme, jakým způsobem mají být data načtena.

- `file`: jaký je název souboru? (`"nazev.zkratkaFormatu"`)
- `header`: je první řádek hlavička? (`TRUE / FALSE`)
- `sep`: co odděluje jednotlivé hodnoty? (`" " "," "\t" ";"`…)
- `dec`: jak určujeme desetinou hodnotu? (`"." "," ";"`…)
- a mnoho jiných…

Když můžeme určit separátor, proč existují specifické funkce?

Nahrání zkušebního souboru s určením argumentů:

```
dataTxt <- read.delim(file = "intro_data.txt",
                      header = TRUE,
                      sep = "", #co je separator
                      dec = ".", #co je desetinna carka
                      na.strings = "NA") #co jsou chybejici hodnoty
```

Nahrání zkušebního souboru bez určení argumentů:

```
dataTxt2 <- read.delim(file = "intro_data.txt")
```

Jsou tyto dva soubory identické?

```
identical(dataTxt, dataTxt2)
```

```
[1] TRUE
```

> **Nezapomeňte na defaultní nastavení arguentů funkce!**
>
> Spousta funkcí má u argumentů nastavené nějaké defaultní (nejčastěji používané) hodnoty.
>
> Je proto zbytečné argumenty vypisovat znovu, pokud se nijak neliší od defaultního

nastavení.

Řešení otázky “proč existují specifické funkce?”

Protože jsme lenoši.

argument `file`

- vždy musí být v uvozovkách a obsahovat přesný název souboru, včetně jeho formátové koncovky
- je potřeba, aby obsahoval **přesný popis cesty** k tomuto souboru
  - Co když máme v projektu dedikovanou **podsložku** na data?

```
1 dataUvnitrPodslozkyvProjektu <- read.delim("data/skrytadata.txt")
```

- Co když si potřebujeme vytáhnout **soubor mimo projekt**, a nebo nemáme projekt ani nastavený working directory?

```
1 dataBezUrcenehoWD <- read.delim("C:/Users/Lucie Hošková/Desktop/introR_CUNI/materi
```

Absolutní a relativní cesta

Nepamatujete si, jak přesně se používají relativní / absolutní cesty a projekty? Podívejte se znovu do první lekce, sekce Working depository a RProjekty.

argument `sep`

Separátor je symbol či série symbolů, které **určují ohraničení jednotlivých sloupců**.

Jaké máme různé druhy separátorů?

- `" "`: white space, aka mezera/mezery
- `","`: coma, čárka - CSV je Coma Separated Values
- `"\t"`: tabulátor
- `";"`: semicolon, středník

Toto jsou nejčastější typy separátorů, **můžete se ovšem setkat i s jinými individualizovanými separátory**.

Detailněji si tato individualizovaná nastavení ukážeme v následující sekci.

argument dec

Pomocí tohoto argumentu si definujeme symboly, které **určují desetinou čárku**.

Je možné používat jako desetinnou čárku libovolný symbol?

vaha	vyska	vek
62X8	165X7	32
75X1	198X1	41
55X9	146X2	13

```
1 decZkouska <- read.delim("decZkouska.txt", dec = "X")
2 head(decZkouska)
```

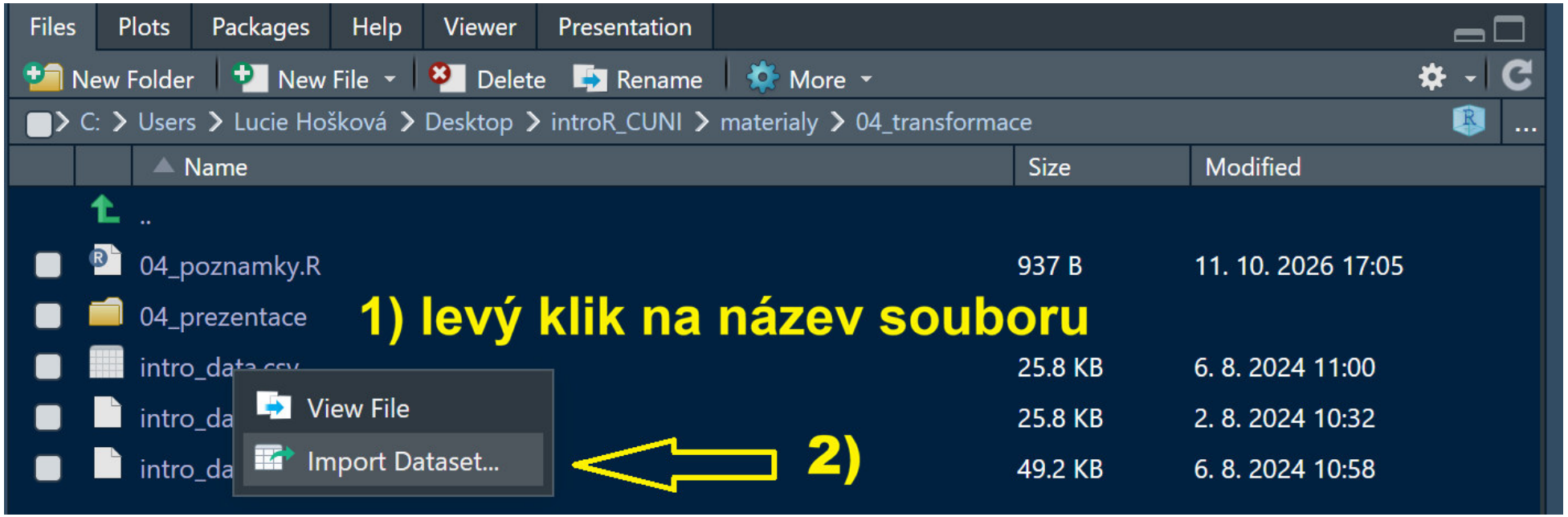
```
  vaha vyska vek
1 62.8 165.7  32
2 75.1 198.1  41
3 55.9 146.2  13
```

Balíček `readr`

- je součástí metabalíčku `tidyverse`
- umožňuje načtení souboru klikáním v RStudiu **bez psaní kódu**
  - pozor, nefunguje na `.txt` data!

Postup:

Vyzkoušíme si to na datech `intro_data.csv`.



Import Text Data

File/URL:

C:/Users/Lucie Hošková/Desktop/introR_CUNI/materialy/04_transformace/intro_data.csv

Update

Data Preview:

ukázka jednotlivých sloupců, jejich hodnot a jakého jsou typu

(rowname	Sex (character)	Side (character)	No (numeric)	D (numeric)	No_sub (numeric)	Age (numeric)	Length_cl (numeric)	Shoulder_w (numeric)	Proportion (numeric)	Inner_C (numeric)	Outer_C (numeric)	Total_C (numeric)	Inner_S (numeric)	Middle_S (numeric)	Outer_S (numeric)	Inner_End_W (numeric)	Inner_A_W (numeric)	Least_W_W (numeric)	Distance1_W (numeric)
1	female	sin	1	1	1	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
2	female	sin	2	0	8	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
3	female	sin	3	0	10	36	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
4	female	sin	4	0	14	59	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
5	female	sin	5	0	20	59	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
6	female	sin	6	0	26	38	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
7	female	sin	7	0	36	65	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA	NA
8	female	sin	8	0	47	52	129	362	356	154	157	311	33	68	29	23	11	8	65
9	female	sin	9	0	60	69	139	357	389	151	155	306	46	68	28	21	10	9	75
10	female	sin	10	0	64	48	141	362	390	154	158	312	64	72	26	16	11	10	29
11	female	sin	11	0	68	20	130	342	380	158	153	311	40	54	86	20	10	10	40
12	female	sin	12	0	73	45	142	367	387	154	156	310	41	71	33	23	10	10	45
13	female	sin	13	0	79	49	149	372	401	146	163	309	53	70	32	23	11	10	67
14	female	sni	14	0	81	46	134	377	355	152	138	290	45	64	28	21	10	10	55
15	female	sin	15	1	82	28	133	330	403	159	154	313	37	67	30	20	8	8	37
16	zena	sin	16	1	83	31	139	385	361	153	159	312	36	71	35	20	11	10	18
17	female	sin	17	1	87	54	145	385	377	158	153	306	48	65	35	22	11	9	30
18	female	sin	18	0	96	46	155	251	441	150	145	295	53	75	84	21	11	10	36
19	female	sin	19	1	98	55	133	333	399	153	152	305	43	63	28	23	11	9	63
20	female	sin	20	1	104	44	127	348	365	156	143	299	38	60	32	21	9	8	48

Dobrá praxe práce s balíčky, které umožňují importování

Vždy si kód zkopírujte do skriptu a manuálně ho projeďte.

- import provedete spuštění kódu na pozadí a ve skriptu po něm nezůstane ani stopa, při příštím spuštění samotného skriptu by vám tak nedošlo k načtení dat

Nezapomeňte si spustit jednotlivé balíčky pomocí fce `library`.

- není třeba nahrávat celý `tidyverse`, stačí jednoduchý název specifického balíčku

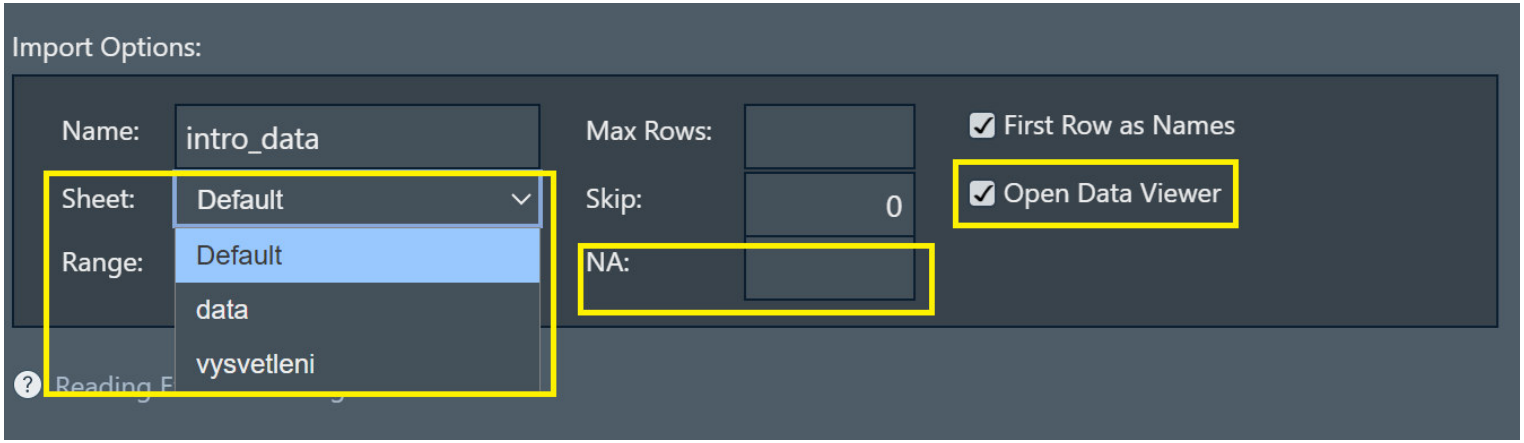
Balíček `readxl` - ach ty Excely…

- je součástí metabalíčku `tidyverse`



Postup je při nahrávání velmi podobný balíčku `readr`, má ovšem **pár důležitých detailů**.

Vyzkoušíme si to na datech `intro_data.xlsx`.



- `Sheet`: z jakého listu chceme vybírat?
- `NA`: narozdíl od `baseR` a `readr` tento balíček neumí automaticky rozpoznávat NA hodnoty
- `Open Data Viewer`: v RStudiu automaticky otevře náhled

Samostatné cvičení

08:00

Načtěte si soubor `samostatneCviceniData.txt` a uložte si ho jako `data`. Přeměňte si sloupec `vzdelani` na faktory[1] a vypočítejte průměrnou výšku ze sloupce `vyska`.

1. Podívejte se do nápovědy; neexistuje na to přímo argument?

Řešení

Načtěte si soubor `samostatneCviceniData.txt` a uložte si ho jako `data`. Přeměňte si sloupec `vzdelani` na faktory a vypočítejte průměrnou výšku ze sloupce `vyska`.

1. Načteme si soubor pomocí funkce `read.delim` a podíváme se na strukturu.

```
1  data <- read.delim("samostatneCviceniData.txt")
2  str(data)
```

```
'data.frame':   10 obs. of  4 variables:
 $ id      : int  26 981 65 84 563 979 646 634 741 753
 $ vzdelani: chr  "ZS" "VS" "ZS" "SS" ...
 $ vek     : int  45 85 17 23 46 18 23 36 65 47
 $ vyska   : chr  "187,8" "174,5" "168,6" "156,6" ...
```

2. Je vidět, že soubor má jiný typ separátoru, než jsme očekávali. Musíme podle toho proto upravit argumenty funkce.

```
1  data <- read.delim("samostatneCviceniData.txt",
2                     dec = ",")
3  str(data)
```

```
'data.frame':   10 obs. of  4 variables:
 $ id      : int  26 981 65 84 563 979 646 634 741 753
 $ vzdelani: chr  "ZS" "VS" "ZS" "SS" ...
 $ vek     : int  45 85 17 23 46 18 23 36 65 47
 $ vyska   : num  188 174 169 157 198 ...
```

Nebo můžeme použít funkci, která má přímo určenu desetinnou čárku.

```
1  data2 <- read.delim2("samostatneCviceniData.txt")
2  str(data2)
```

```
'data.frame':   10 obs. of  4 variables:
 $ id      : int  26 981 65 84 563 979 646 634 741 753
 $ vzdelani: chr  "ZS" "VS" "ZS" "SS" ...
 $ vek     : int  45 85 17 23 46 18 23 36 65 47
 $ vyska   : num  188 174 169 157 198
```

3. Změňme charakterové stringy (sloupec `vzdelani`) na faktory.

```
data <- read.delim("samostatneCviceniData.txt",
                   dec = ",",
                   stringsAsFactors = TRUE)
str(data)
```

```
'data.frame':   10 obs. of  4 variables:
 $ id      : int  26 981 65 84 563 979 646 634 741 753
 $ vzdelani: Factor w/ 3 levels "SS","VS","ZS": 3 2 3 1 2 3 1 1 1 1
 $ vek     : int  45 85 17 23 46 18 23 36 65 47
 $ vyska   : num  188 174 169 157 198 ...
```

4. Spočítejme si průměrnou výšku.

```
mean(data$vyska) #175.15
```

```
[1] 175.15
```

Transformace dat

Máme nahrána data, co s nimi ted´?

Před tím, než se vrhneme na jakoukoliv analýzu, tak **je třeba si data zkontrolovat a dle potřeby upravit**.



Podivné názvy sloupců

Někdy se může stát, že máte v R dataset, u které jsou jsou **sloupce s totožnými názvy následované čísly a s tečkami mezi slovy**, i když v původním souboru nic takového není.

```
df <- read.delim("bydliste.txt")
head(df)
```

```
  gender vek pocet.deti misto.bydliste rodinny.stav misto.bydliste.1
1      F  19          0         Turnov          N/A       Veselská 5
2      M  85          4        Všenory   manželství        Malá 56/7
3      X  54          2        Karviná  partnerství    U hangáru 898
4      M N/A          3           Zlín     svobodní                 
5      M  22          0           Most     svobodní                 
6      F  18          1          Louny          N/A              N/A
```

```
str(df)
```

```
'data.frame':   15 obs. of  6 variables:
 $ gender          : chr  "F" "M" "X" "M" ...
 $ vek             : chr  "19" "85" "54" "N/A" ...
 $ pocet.deti      : chr  "0" "4" "2" "3" ...
```

```
$ misto.bydliste  : chr  "Turnov" "Všenory" "Karviná" "Zlín" ...
$ rodinny.stav    : chr  "N/A" "manželství" "partnerství" "svobodní" ...
$ misto.bydliste.1: chr  "Veselská 5" "Malá 56/7" "U hangáru 898" "" ...
```

Co se to děje?

R automaticky kontroluje, zda **jsou všechny názvy sloupců "validní"**. Provádí se to pomocí defaultního argumentu fce `read.table` a to `check.names = TRUE`.

To znamená, že nemají mezery a **slova jsou oddělena stejným symbolem**, a v **případě duplikátů jmen sloupců** je k nim přiřazen symbol, který je odliší.

Co když se nám ale dané jméno nelíbí? Jak si ho přejmenovat?

```
1 df <- df %>%
2   dplyr::rename(adresa = misto.bydliste.1)
3 head(df)
```

```
  gender vek pocet.deti misto.bydliste rodinny.stav        adresa
1      F  19          0         Turnov          N/A    Veselská 5
2      M  85          4        Všenory   manželství     Malá 56/7
3      X  54          2        Karviná  partnerství U hangáru 898
4      M N/A          3           Zlín     svobodní
5      M  22          0           Most     svobodní
6      F  18          1          Louny          N/A           N/A
```

Přejmenování bez potřeby znát pozici sloupce

Sice už umíme používat fce `dimnames`, `colnames` a jejich příslušné varianty v argumentech funkcí, i tak se ale hodí znát funkci, která nám **změní specifický sloupec bez toho, abychom museli hledat jeho pořadí** či nějak složitě vypisovat veškerá jména sloupců.

Chybějící hodnoty

Při kontrole dat jsme si mohli všimnout, že v souboru jsou podivně zapsané chybějící hodnoty - jak prázdná místa, tak text `N/A`.

Jak toto vše můžeme převést na jednoduchá `NA`?

```
1 dfNA <- read.delim2("bydliste.txt",
2                     na.strings = c("", "N/A")) #"" urcuje mezeru/prazdne pole
3 dfNA
```

```
   gender vek pocet.deti misto.bydliste rodinny.stav      misto.bydliste.1
1       F  19          0         Turnov         <NA>             Veselská 5
2       M  85          4       Všenory   manželství              Malá 56/7
3       X  54          2       Karviná  partnerství          U hangáru 898
4       M  NA          3          Zlín     svobodní                  <NA>
5       M  22          0          Most     svobodní                  <NA>
6       F  18          1         Louny         <NA>                  <NA>
7       F  23          0         Opava     svobodní           Novákové 13
8       F  36          2         Plzeň     svobodní         Destinové 42
9       X  68          3         Jičín   manželství                  <NA>
```

```
10      M  47          1        Trutnov  partnerství             Buková 65
11   <NA>  NA         NA           <NA>        <NA>                  <NA>
12      X  33          1          Kuřim        <NA>           U topolu 78
```

Nejlepším místem je již určení si hodnot při samotném načítání dat argumentem `na.strings`. Někdy se ale může stát, že si toho všimnete pozdě, a už jste na datasetu provedli rozličné změny.

Jak si tedy můžete tyto hodnoty změnit zpětně?

Samostatné cvičení

05:00

Upravte hodnoty uvnitř data frame `df` tak, aby místo prázdných buněk a charakterové hodnoty `N/A` byly přímo hodnoty `NA`.

> **Nezapomeňte na to, co jsme se již naučili**
>
> Na předchozí lekci jsme si ukázali, jakým způsobem lze vybrat z celého data frame specifické hodnoty. Jak bychom to mohli aplikovat zde?
>
> **Přemýšlejte o tom jednoduše - není třeba žádných složitých indexací.**

Řešení

Upravte hodnoty uvnitř data frame `df` tak, aby místo prázdných buněk a charakterové hodnoty `N/A` byly přímo hodnoty `NA`.

1. Pomocí základní indexace si vybereme všechny hodnoty z `df`, pro které platí `df == "N/A" | df == ""`.

```
1  df == "N/A" | df == ""
```

```
      gender   vek pocet.deti misto.bydliste rodinny.stav adresa
 [1,]  FALSE FALSE      FALSE          FALSE         TRUE  FALSE
 [2,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
 [3,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
 [4,]  FALSE  TRUE      FALSE          FALSE        FALSE   TRUE
 [5,]  FALSE FALSE      FALSE          FALSE        FALSE   TRUE
 [6,]  FALSE FALSE      FALSE          FALSE         TRUE   TRUE
 [7,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
 [8,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
 [9,]  FALSE FALSE      FALSE          FALSE        FALSE   TRUE
[10,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
[11,]   TRUE  TRUE       TRUE           TRUE         TRUE   TRUE
[12,]  FALSE FALSE      FALSE          FALSE         TRUE  FALSE
[13,]  FALSE FALSE      FALSE          FALSE        FALSE  FALSE
```

2. Tyto hodnoty si přepíšeme na `NA`

```
1  df[df == "N/A" | df == ""] <- NA;
2  df
```

```
  gender  vek pocet.deti misto.bydliste rodinny.stav                adresa
```

Odstranění NA hodnot

Dejme ale, že bychom v datasetu chtěli mít pouze kompletní řádky, tzn. chceme se zbavit řádků obsahující NA hodnoty. Jak na to?

```
1 dim(df)
```

```
[1] 15  6
```

```
1 df <- na.omit(df)
2 dim(df)
```

```
[1] 7 6
```

```
1 df
```

	gender	vek	pocet.deti	misto.bydliste	rodinny.stav	adresa
2	M	85	4	Všenory	manželství	Malá 56/7
3	X	54	2	Karviná	partnerství	U hangáru 898
7	F	23	0	Opava	svobodní	Novákové 13
8	F	36	2	Plzeň	svobodní	Destinové 42
10	M	47	1	Trutnov	partnerství	Buková 65
13	M	44	4	Kladno	partnerství	Dolanská 70
15	F	76	2	Brno	manželství	Mendlovo náměstí 8

Co se stalo? Zmizely všechny řádky, co měli v jakémkoliv sloupci NA. O jaké řádky se ale jednalo?

```
  1  which(!complete.cases(dfNA))
```

```
[1]  1  4  5  6  9 11 12 14
```

Dokážete vysvětlit, co se v tomto kombinací těchto funkcí a jejich aplikaci na dataset stane?

Co dělá funkce `complete.cases` a proč ji zde používáme?

1. **Jako první se podíváme do nápovědy** a zjistíme si, co znamená funkce `complete.cases`.
2. Ta vytvoří vektor, který v sobě bude obsahovat **informace, zda jednotlivé řádky jsou kompletní** - neobsahují žádné chybějící hodnoty.
3. My ovšem chceme vědět, které řádky chybějící hodnoty obsahují - **provedeme proto negaci pomocí symbolu** `!`.
4. A funkce `which` nám pak už pouze řekne, na jaké pozici ve vektoru těchto hodnot se výsledné `TRUE` (řádek obsahuje `NA` hodnoty) nachází.

Máme nyní přejmenované sloupce, odebrané NA hodnoty… Znamená to, že s ním nyní můžeme provádět výpočty, či na něj aplikovat jakékoliv další akce?

```
1 str(df)
```

```
'data.frame':   7 obs. of  6 variables:
 $ gender        : chr  "M" "X" "F" "F" ...
 $ vek           : chr  "85" "54" "23" "36" ...
 $ pocet.deti    : chr  "4" "2" "0" "2" ...
 $ misto.bydliste: chr  "Všenory" "Karviná" "Opava" "Plzeň" ...
 $ rodinny.stav  : chr  "manželství" "partnerství" "svobodní" "svobodní" ...
 $ adresa        : chr  "Malá 56/7" "U hangáru 898" "Novákové 13" "Destinové 42" ...
 - attr(*, "na.action")= 'omit' Named int [1:8] 1 4 5 6 9 11 12 14
  ..- attr(*, "names")= chr [1:8] "1" "4" "5" "6" ...
```

Jak vidíme, **kvůli špatně zapsaným NA hodnotám se nám většina sloupců překonvertovala do charakterového typu**.

Když se ale na data frame podíváme, tak je jasné, že některé sloupce můžeme změnit na faktory a jiné budou jasně

numeric.

```
1 df
```

	gender	vek	pocet.deti	misto.bydliste	rodinny.stav	adresa
2	M	85	4	Všenory	manželství	Malá 56/7
3	X	54	2	Karviná	partnerství	U hangáru 898
7	F	23	0	Opava	svobodní	Novákové 13
8	F	36	2	Plzeň	svobodní	Destinové 42
10	M	47	1	Trutnov	partnerství	Buková 65
13	M	44	4	Kladno	partnerství	Dolanská 70
15	F	76	2	Brno	manželství	Mendlovo náměstí 8

Funkce `mutate` aneb úprava a vytváření sloupců

Pomocí funkce `mutate` můžeme upravovat specifické sloupce bez dlouhého přiřazování.

Při jejím používání defacto potřebujeme pouze následující zápis:

`mutate(název dataframe, název sloupce = nová podoba sloupce)`

```
df <- df %>% mutate(
  gender = as.factor(gender),
  vek = as.numeric(vek),
  pocet.deti = as.numeric(pocet.deti),
  rodinny.stav = as.factor(rodinny.stav)
)
```

Jaký je nyní výsledek?

```
1 str(df)
```

```
'data.frame':   7 obs. of  6 variables:
 $ gender        : Factor w/ 3 levels "F","M","X": 2 3 1 1 2 2 1
 $ vek           : num  85 54 23 36 47 44 76
 $ pocet.deti    : num  4 2 0 2 1 4 2
 $ misto.bydliste: chr  "Všenory" "Karviná" "Opava" "Plzeň" ...
 $ rodinny.stav  : Factor w/ 3 levels "manželství","partnerství",..: 1 2 3 3 2 2 1
 $ adresa        : chr  "Malá 56/7" "U hangáru 898" "Novákové 13" "Destinové 42" ...
 - attr(*, "na.action")= 'omit' Named int [1:8] 1 4 5 6 9 11 12 14
  ..- attr(*, "names")= chr [1:8] "1" "4" "5" "6" ...
```

Přidávání sloupců

Tímto způsobem si ale můžeme sloupce nejenom upravovat, ale i přidávat.

```
df <- df %>% mutate(
  telefonni.cislo = NA,
  id = paste0("id_", 1:nrow(df)))
head(df)
```

```
   gender vek pocet.deti misto.bydliste rodinny.stav        adresa
2       M  85          4        Všenory   manželství     Malá 56/7
3       X  54          2        Karviná  partnerství U hangáru 898
7       F  23          0          Opava     svobodní   Novákové 13
8       F  36          2          Plzeň     svobodní  Destinové 42
10      M  47          1        Trutnov  partnerství     Buková 65
13      M  44          4         Kladno  partnerství   Dolanská 70
   telefonni.cislo   id
2               NA id_1
3               NA id_2
7               NA id_3
8               NA id_4
10              NA id_5
13              NA id_6
```

Samostatné cvičení

25:00

Proved na souboru `census.csv` následující změny:

1. Přejmenuj sloupce `pohlavi_txt` na `pohlavi`, `vek_txt` na `vek`, `vira_txt` na `vira`, a `uzemi_txt` na `oblast`.
2. Vyber pouze hodnoty pohlaví `žena`, ty kde `vira` je `Jedi`, `Sith` a nebo začíná na `pasta`[1] a nakonec pak pouze ty osoby, které mají věk v rozpětí 0-18 a 65+[2].
3. Vyber pouze hodnoty pro celý stát, tedy ty, které mají ve sloupci `oblast` hodnotu `Česká republika`.
4. Vyber sloupce `vek`, `pohlavi`, `vira` a `hodnota`.
5. Kolik je nejstarší české Jedi? Zjisti funkcí.
6. Spočítej pro každý kraj průměrný věk pro všechna náboženství.

1. Něco podobného jsme prováděli u funkce `select`. Existuje nějaká funkce z `base`, která lze použít?

Řešení

1. Načti data a zkontroluj je.

```
census <- read.csv("census.csv")
str(census)
```

```
'data.frame':   58890 obs. of  19 variables:
 $ idhod       : int  989095533 989113507 989113508 989096570 989112602 989112603
989097795 989116863 989116864 989099080 ...
 $ hodnota     : int  51397 26279 25118 52693 26922 25771 54488 28101 26387 55657
...
 $ ukaz_kod    : int  3162 3162 3162 3162 3162 3162 3162 3162 3162 3162 ...
 $ vira_cis    : int  3078 3078 3078 3078 3078 3078 3078 3078 3078 3078 ...
 $ vira_kod    : int  1 1 1 1 1 1 1 1 1 1 ...
 $ vira_cleneni: int  1 1 1 1 1 1 1 1 1 1 ...
 $ vek_cis     : int  1035 1035 1035 1035 1035 1035 1035 1035 1035 1035 ...
 $ vek_kod     : int  1000000000 1000000000 1000000000 1000010001 1000010001
1000010001 1000020002 1000020002 1000020002 1000030003 ...
 $ pohlavi_cis : int  NA 102 102 NA 102 102 NA 102 102 NA ...
 $ pohlavi_kod : int  NA 1 2 NA 1 2 NA 1 2 NA ...
```

2. Přejmenuj sloupce

```
census <- census %>%
  rename(
    vira = vira_txt,
    pohlavi = pohlavi_txt,
```

A nešlo by to nějak… jednodušeji?

Samozřejmě :]

Vytvoření datasetu na kterém budeme vytvářet výpočty si jde napsat pomocí dlouhé pipe.

```
1 census2 <- read.csv("census.csv")
2 str(census2)
```

```
'data.frame':   58890 obs. of  19 variables:
 $ idhod       : int  989095533 989113507 989113508 989096570 989112602 989112603
989097795 989116863 989116864 989099080 ...
 $ hodnota     : int  51397 26279 25118 52693 26922 25771 54488 28101 26387 55657 ...
 $ ukaz_kod    : int  3162 3162 3162 3162 3162 3162 3162 3162 3162 3162 ...
 $ vira_cis    : int  3078 3078 3078 3078 3078 3078 3078 3078 3078 3078 ...
 $ vira_kod    : int  1 1 1 1 1 1 1 1 1 1 ...
 $ vira_cleneni: int  1 1 1 1 1 1 1 1 1 1 ...
 $ vek_cis     : int  1035 1035 1035 1035 1035 1035 1035 1035 1035 1035 ...
 $ vek_kod     : int  1000000000 1000000000 1000000000 1000010001 1000010001
1000010001 1000020002 1000020002 1000020002 1000030003 ...
 $ pohlavi_cis : int  NA 102 102 NA 102 102 NA 102 102 NA ...
 $ pohlavi_kod : int  NA 1 2 NA 1 2 NA 1 2 NA ...
 $ uzemi_cis   : int  97 97 97 97 97 97 97 97 97 97 ...
```

```r
census2 <- census2 %>%
  rename(
    vira = vira_txt,
    pohlavi = pohlavi_txt,
    vek = vek_txt,
    oblast = uzemi_txt) %>%
  mutate(vek = readr::parse_number(vek)) %>%
  filter(
    vira %in% c("Jedi", "Sith") | startsWith(vira, "pasta"),
    pohlavi == "žena",
    vek %in% 0:18 | vek >= 65,
    oblast == "Česká republika",
    hodnota > 0) %>%
  select(hodnota, pohlavi, vek, vira)
```

Poté už jen provedeme výpočet jednotlivých hodnot.

```r
census2 %>% group_by(vira) %>%
  summarize(nejstarsiVerici = max(vek), prumernyVek = mean(vek))
```

```
# A tibble: 3 × 3
  vira            nejstarsiVerici prumernyVek
  <chr>                     <dbl>       <dbl>
1 Jedi                         98        50.0
2 Sith                         68        14.5
3 pastafariánství              84        32.6
```

S funkcemi `group_by` a `summarize` se lépe seznámíme v šesté sekci, zde jsou uvedeny jako ukázka zjednodušení postupu.

Domácí úkol

1. Najděte specifickou funkci v balíčku `dplyr`, která umožňuji přepisování hodnot na `NA`.
2. Načti si dataset `typBydleni.csv`. Zjisti z něj následující údaje:
    1. Kolik osob ve věku 18-26 včetně v Jihomoravském kraji jsou evidováni jako osoby bez domova, a jakou procentuální část z nich tvoří muži a jakou ženy.
    2. Který kraj má nejmenší počet obyvatel žijících v ubytovacích zařízení?
    3. Jaký je medián věku osoby bez domova v Praze, a jaký je v celé ČR?
    4. Jaký je nejčastější typ ubytování osob pod 18 let v ČR?
    5. O kolik více seniorů (65+ let) bez domova žije v

Tipy navíc

Budou doplněny během dneška.

Změna pořadí sloupců

Co kdybychom chtěli mít sloupce na jiné pozici, než jsou v nahraném datasetu?

Použijeme k tomu funkci `dplyr::relocate`.

```
dfTipy <- read.delim("intro_data.txt")
dfTipy <- dfTipy[,1:7] #ukazkovy dataset

dfTipy <- dfTipy %>% relocate(No:No_sub, .before = Sex)
dfTipy <- dfTipy %>% relocate(Sex, .after = Age)
dfTipy <- dfTipy %>% relocate(Side, .after = last_col())
```

Seřazení řádků dle hodnot ve sloupcích

Může se stát, že si budete potřebovat srovnat podle hodnot v nějakém sloupci. Jak to provedete?

```
1 dfTipy %>% arrange(Age) #vzestupne
```

```
   No D No_sub  Age    Sex Length_cl Side
1   58 1    176   18 female       134  sin
2   58 1    176   18 female       132   dx
3   26 1     66   18   male        NA  sin
4   17 1     66   18   male       126   dx
5   28 1    112   19 female       132  sin
6   50 1    153   19 female       142  sin
7   29 1    112   19 female       132   dx
8   50 1    153   19 female       142   dx
9   48 1    102   19   male       158  sin
10  76 1    166   19   male       144  sin
11  37 1    102   19   male       160   dx
12  63 1    166   19   male        NA   dx
13  11 0     68   20 female       130  sin
```

```
1 dfTipy %>% arrange(desc(Age)) #sestupne
```

```
   No D No_sub  Age    Sex Length_cl Side
```

1	38	1	90	old	male	140	sin
2	28	1	90	old	male	140	dy
3	60	1	179	76	female	146	sin
4	61	1	179	76	female	148	dx
5	24	1	108	73	female	139	sin
6	37	1	139	73	female	140	sin
7	25	1	108	73	female	135	dx
8	38	1	139	73	female	139	dx
9	38	1	140	72	female	137	sin
10	39	1	140	72	female	137	dx
11	39	1	91	72	male	157	sin
12	29	1	91	72	male	153	dx
13	54	1	118	71	male	153	sin

Co když ale máme ve sloupci více stejných hodnot? Můžeme si určit další sloupec, dle kterého by se mělo filtrovat?

Přidáváním dalších a dalších sloupců, dle kterých se může srovnávat, si můžeme vytvořit opravdu podrobně seřazený data frame.

```
1  dfTipy %>% arrange(Age, Length_cl)
```

```
    No D No_sub  Age    Sex Length_cl Side
1   17 1     66   18   male       126   dx
2   58 1    176   18 female       132   dx
3   58 1    176   18 female       134  sin
4   26 1     66   18   male        NA  sin
5   28 1    112   19 female       132  sin
6   29 1    112   19 female       132   dx
7   50 1    153   19 female       142  sin
8   50 1    153   19 female       142   dx
9   76 1    166   19   male       144  sin
10  48 1    102   19   male       158  sin
11  37 1    102   19   male       160   dx
12  63 1    166   19   male        NA   dx
13  11 0     68   20 female       130  sin
```

Načítání velkých souborů

V balíčku `readr` se nachází taktéž funkce `read_csv`.

Tu se hodí používat, pokud nahráváte objemnější datasety; běží 2-3x rychleji než `read.csv` v základním R.